

Python & React Interview Preparation Guide

For Java Developers Expanding Their Stack

12
Python Q&A

12
React Q&A


Java bridges

- ✓ Every question answered from a Java developer's perspective
- ✓ Code examples for every concept
- Java parallel highlights — map familiar concepts to new syntax
- ✓ Python core language, OOP, async, packaging
- ✓ React hooks, state management, routing, TypeScript, performance

Table of Contents

■ Python Section

- P1. How is Python different from Java in terms of typing and compilation?
- P2. Explain Python's indentation rule and how it replaces braces.
- P3. What are Python lists, tuples, sets, and dicts? How do they map to Java col...
- P4. What is a Python list comprehension and when should you use it?
- P5. Explain Python functions as first-class objects, lambdas, and decorators.
- P6. What is the difference between *args and **kwargs?
- P7. How does Python handle exceptions? What's different from Java?
- P8. What are Python generators and when would you use them over lists?
- P9. Explain Python's OOP: classes, inheritance, and dunder methods.
- P10. What are Python's context managers and the `with` statement?
- P11. What is the GIL and how does it affect multi-threading in Python?
- P12. How do you manage packages and virtual environments in Python?

■■ React Section

- R1. What is React and how does it differ from a traditional MVC framework?
- R2. What is JSX and how does it compile?
- R3. What is the difference between props and state?
- R4. Explain React hooks: useState, useEffect, useRef, useMemo.
- R5. What is the virtual DOM and how does React's reconciliation work?
- R6. How does component communication work? (Parent↔Child, siblings, global stat...
- R7. How do you fetch data from an API in React?
- R8. What is event handling in React? How is it different from plain HTML?
- R9. What is React Router and how does client-side routing work?
- R10. How do you handle forms in React? Controlled vs uncontrolled components.
- R11. What are common React performance optimisations?
- R12. How does TypeScript improve React development?

Reading Guide

Q#	Question number
Answer	Explanation of the concept
Code block	Runnable code example
■ Java parallel	How this maps to Java/Spring you already know

■ PYTHON

12 Questions — Core Language, OOP, Concurrency, Packaging

QP1 How is Python different from Java in terms of typing and compilation?

Answer

Python is **dynamically typed** — types are checked at runtime, not compile time. You never declare variable types; the interpreter infers them. Java is statically typed with compile-time checks.

Python is also **interpreted** (CPython compiles to bytecode then interprets), whereas Java compiles to JVM bytecode that the JVM JIT-compiles. This makes Python generally slower but faster to prototype.

```
# Python — no type declarations needed
x = 42 # int inferred
x = 'hello' # now str — perfectly legal
# Optional type hints (Python 3.5+) — not enforced at runtime
def add(a: int, b: int) -> int:
    return a + b
```

■ *Java parallel: `int x = 42`; in Java is fixed to `int`. Python's equivalent is just `x = 42`.*

QP2 Explain Python's indentation rule and how it replaces braces.

Answer

Python uses **indentation (whitespace) to define code blocks** instead of curly braces. The standard is 4 spaces per level. Mixing tabs and spaces causes `IndentationError`.

Every block (if, for, def, class, try) must be consistently indented. The colon `:` at the end of a statement signals the start of a new block.

```
def check_positive(n): # colon starts a block
    if n > 0: # 4-space indent
        print('positive') # 8-space indent
    else:
        print('non-positive') # same 8-space level
    return n # back to 4-space block
```

■ *Java parallel: Java's {} are replaced by : + indentation. The colon is mandatory.*

QP3 What are Python lists, tuples, sets, and dicts? How do they map to Java collections?

Answer

list → `ArrayList<Object>`: ordered, mutable, allows duplicates. **tuple** → immutable list (no Java direct equivalent, think unmodifiable list). **set** → `HashSet`: unordered, unique elements. **dict** → `HashMap`: key-value pairs.

```
lst = [1, 2, 2, 3] # list – ordered, mutable
tpl = (1, 2, 3) # tuple – ordered, immutable
st = {1, 2, 3} # set – unordered, unique
d = {'key': 'value'} # dict – key-value pairs
# Common operations
lst.append(4) # add to end
lst.pop() # remove last
d['new_key'] = 99 # insert/update
'key' in d # True – membership check
```

■ *Java parallel: `ArrayList.add()`, `HashMap.put()`, `HashSet.add()` — same concepts, cleaner syntax.*

QP4 What is a Python list comprehension and when should you use it?

Answer

A **list comprehension** is a concise one-liner to create a list by iterating and optionally filtering. It replaces the Java pattern of: create list → for-loop → add elements.

Use it for simple transformations and filters. Avoid it for complex multi-step logic — keep it readable.

```
numbers = [1, 2, 3, 4, 5, 6]
# Java: new ArrayList then for-loop
# Python equivalents:
squares = [x**2 for x in numbers]
evens = [x for x in numbers if x % 2 == 0]
even_squares = [x**2 for x in numbers if x % 2 == 0]
# Dict comprehension
sq_map = {x: x**2 for x in range(5)} # {0:0, 1:1, 2:4, 3:9, 4:16}
# Set comprehension
unique_lens = {len(w) for w in ['hi', 'hello', 'hey']} # {2, 5, 3}
```

■ *Java parallel: Java streams: `numbers.stream().filter(...).map(...).collect(toList())` — same idea, different syntax.*

QP5 Explain Python functions as first-class objects, lambdas, and decorators.

Answer

In Python, **functions are objects** — they can be passed as arguments, returned from other functions, and stored in variables. **lambda** creates anonymous single-expression functions.

Decorators are functions that wrap other functions to add behaviour (logging, auth, caching). They use the `@decorator` syntax — equivalent to Java's AOP or annotation-based proxies (like Spring's `@Transactional`).

```
# Lambda (anonymous function)
double = lambda x: x * 2
print(double(5)) # 10
# Functions as arguments
nums = [3, 1, 4, 1, 5]
nums.sort(key=lambda x: -x) # sort descending
# Decorator
def logger(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        result = func(*args, **kwargs)
        print(f'Done')
        return result
    return wrapper
@logger
def add(a, b):
    return a + b
add(2, 3) # prints: Calling add / Done / returns 5
```

■ *Java parallel: Java lambdas: `(x) -> x*2`. Python's `@decorator` ≈ Spring AOP proxies or `@Around` advice.*

QP6 What is the difference between `*args` and `**kwargs`?

Answer

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They allow functions to accept a variable number of arguments — like Java's `varargs`String... args`` but more flexible.

```
def greet(*args, **kwargs):
    for name in args:
        print(f'Hello {name}') # positional
    for k, v in kwargs.items():
        print(f'{k} = {v}') # keyword
    greet('Alice', 'Bob', city='NYC', lang='Python')
# Hello Alice
# Hello Bob
# city = NYC
# lang = Python
```

■ *Java parallel: Java `varargs: void greet(String... names)`. Python `**kwargs` adds named parameters on top.*

QP7 How does Python handle exceptions? What's different from Java?

Answer

Python uses `try/except/else/finally`. Key differences from Java: no checked exceptions (nothing forces you to catch); you can catch multiple exception types in one `except``; the `else`` clause runs only if no exception was raised.

```
try:
    result = 10 / int(input('Enter number: '))
except ZeroDivisionError:
    print('Cannot divide by zero')
except ValueError as e:
    print(f'Bad input: {e}')
except (TypeError, OverflowError) as e: # multiple types
    print(f'Math error: {e}')
else:
    print(f'Result: {result}') # runs only if no exception
finally:
    print('Always runs') # like Java finally
# Raising custom exceptions
class InsufficientFundsError(Exception):
    def __init__(self, amount):
        super().__init__(f'Need ${amount} more')
```

■ *Java parallel: Java checked exceptions (`IOException`) don't exist in Python — all exceptions are like `RuntimeException`.*

QP8 What are Python generators and when would you use them over lists?

Answer

A **generator** is a function using `yield` that returns an iterator producing values lazily — one at a time, only when requested. Unlike a list, it doesn't store all values in memory.

Use generators for: large datasets (reading a 10GB file line by line), infinite sequences, pipeline processing. They are memory-efficient because items are computed on demand.

```
# List – stores ALL values in memory
squares_list = [x**2 for x in range(1_000_000)] # uses ~8MB
# Generator – computes lazily, uses ~200 bytes
def squares_gen(n):
    for x in range(n):
        yield x**2 # suspends here, resumes on next()
gen = squares_gen(1_000_000)
print(next(gen)) # 0
print(next(gen)) # 1
# Generator expression (like list comprehension but lazy)
gen2 = (x**2 for x in range(1_000_000))
# Reading a huge file efficiently
def read_large_file(path):
    with open(path) as f:
        for line in f: # generator under the hood
            yield line.strip()
```

■ *Java parallel: Java equivalent: Iterable with custom Iterator, or Java 8 Streams (lazy evaluation).*

QP9 Explain Python's OOP: classes, inheritance, and dunder methods.

Answer

Python is fully OOP. Key differences from Java: no access modifiers (use ``_`` prefix for convention-private); single and multiple inheritance; **dunder (double-underscore) methods** like ``__init__``, ``__str__``, ``__len__`` are Python's way of operator overloading and interface compliance.

```
class Animal:
    def __init__(self, name: str): # like constructor
        self.name = name # instance variable
        self._secret = 'private by convention'
    def __str__(self): # like toString()
        return f'Animal({self.name})'
    def __repr__(self): # dev-friendly string
        return f'Animal(name={self.name!r})'
    def speak(self): # like abstract method
        raise NotImplementedError
class Dog(Animal): # inheritance
    def speak(self): # override
        return 'Woof!'
d = Dog('Rex')
print(d) # Animal(Rex) via __str__
print(d.speak()) # Woof!
```

■ *Java parallel: ``__init__`` = constructor, ``__str__`` = `toString()`, ``_name`` = private field. `super().__init__()` = `super()`.*

QP1 What are Python's context managers and the `with` statement?

0

Answer

A **context manager** guarantees setup and teardown of resources — files, DB connections, locks — even if an exception occurs. It implements `__enter__` and `__exit__`. The `with` statement is Python's try-with-resources (Java 7+).

```
# File handling — auto-closes even on exception
with open('data.txt', 'r') as f:
    content = f.read() # f.close() called automatically
# Custom context manager
from contextlib import contextmanager
@contextmanager
def timer(label):
    import time
    start = time.time()
    try:
        yield # body of 'with' runs here
    finally:
        elapsed = time.time() - start
        print(f'{label}: {elapsed:.3f}s')
    with timer('My operation'):
        [x**2 for x in range(100_000)]
    # My operation: 0.012s
```

■ *Java parallel: Java try-with-resources: `try (FileReader f = new FileReader(path)) { ... }` — same guarantee.*

QP1 What is the GIL and how does it affect multi-threading in Python?

1

Answer

The **Global Interpreter Lock (GIL)** is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This means Python threads do NOT achieve true parallelism for CPU-bound work.

I/O-bound tasks (HTTP requests, file reads) still benefit from threading because the GIL is released during I/O waits. For CPU-bound parallelism use **multiprocessing** (separate processes, each with own GIL) or libraries like NumPy that release the GIL internally.

```
# I/O-bound → threading is fine
import threading, requests
urls = ['https://api.example.com/1', 'https://api.example.com/2']
threads = [threading.Thread(target=requests.get, args=(u,)) for u in urls]
[t.start() for t in threads]
[t.join() for t in threads]

# CPU-bound → use multiprocessing
from multiprocessing import Pool
def crunch(n): return sum(i*i for i in range(n))
with Pool(4) as p:
    results = p.map(crunch, [10**6]*4) # 4 cores, true parallelism
# Or use concurrent.futures (cleaner API)
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
with ThreadPoolExecutor(max_workers=10) as ex:
    futures = [ex.submit(requests.get, u) for u in urls]
```

■ *Java parallel: Java's Thread doesn't have a GIL — all threads share heap freely. In Python, default threading ≈ Java's single-threaded executor for CPU work.*

QP1 How do you manage packages and virtual environments in Python?

2

Answer

Python uses **pip** for package management and **virtual environments** to isolate project dependencies. A virtualenv is a self-contained directory with its own Python and packages — preventing version conflicts between projects.

requirements.txt or **pyproject.toml** (modern) list dependencies — equivalent to Java's pom.xml (Maven) or build.gradle.

```
# Create and activate virtual environment
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate # Windows
# Install packages
pip install requests flask sqlalchemy
# Freeze dependencies
pip freeze > requirements.txt
# Install from requirements
pip install -r requirements.txt
# Modern: use pyproject.toml with poetry or uv
# poetry add requests
# poetry install
```

■ *Java parallel: pip = Maven/Gradle. requirements.txt = pom.xml. venv = Maven's local repo isolation per project.*

■ ■ REACT

12 Questions — Components, Hooks, State, Routing, TypeScript

QR1 What is React and how does it differ from a traditional MVC framework?

Answer

React is a **JavaScript/TypeScript library for building UIs** using a component tree. Unlike traditional MVC (Spring MVC, Angular), React focuses solely on the **View** layer. It uses a **virtual DOM** to efficiently diff and update only changed parts of the real DOM.

React is **declarative** — you describe what the UI should look like for a given state, and React handles the DOM updates. This contrasts with imperative DOM manipulation (jQuery-style).

```
// Declarative: describe the UI based on state
function Counter() {
  const [count, setCount] = React.useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
// React handles DOM updates — you don't touch it directly
```

■ *Java parallel: Think of a React component like a Java method that returns HTML. State = instance fields. Props = constructor arguments.*

QR2 What is JSX and how does it compile?

Answer

JSX (JavaScript XML) is syntax sugar that lets you write HTML-like code inside JavaScript/TypeScript. It is NOT HTML — it compiles to `React.createElement()` calls via Babel/TypeScript compiler.

Key differences from HTML: use `className` instead of `class`; `htmlFor` instead of `for`; all tags must be closed; JavaScript expressions go inside `{}`.

```
// JSX you write:
const element = <h1 className='title'>Hello, {name}!</h1>;
// What it compiles to:
const element = React.createElement('h1',
  { className: 'title' },
  'Hello, ', name, '!'
);
// JSX rules:
// 1. Single root element (or use <>...</> Fragment)
// 2. className, not class
// 3. Self-close empty tags: <img /> <br />
// 4. JS expressions: {2 + 2} {user.name} {isLoggedIn && <Nav />}
```

■ *Java parallel: JSX ≈ Java's Thymeleaf/Freemarker template, but it lives in the same file as your component logic.*

QR3 What is the difference between props and state?

Answer

Props (properties) are **read-only inputs** passed from parent to child — like constructor arguments. A child cannot modify its own props. **State** is **private mutable data** owned by the component — like instance fields. When state changes, React re-renders the component.

```
// Props – read-only, passed from parent
function Greeting({ name, age }) { // destructured props
  return <h1>Hi {name}, you are {age}</h1>;
}
// Parent passes props
<Greeting name='Musthafa' age={30} />
// State – owned by the component, mutable
function Counter() {
  const [count, setCount] = React.useState(0); // [value, setter]
  const [user, setUser] = React.useState({ name: '', age: 0 });
  return (
    <button onClick={() => setCount(prev => prev + 1)}>
      Clicked {count} times
    </button>
  );
}
```

■ *Java parallel: props = constructor params (final fields). state = mutable instance fields. setState() triggers re-render like Spring MVC model update.*

QR4 Explain React hooks: useState, useEffect, useRef, useMemo.

Answer

Hooks let functional components use React features previously only in class components. **useState**: local state. **useEffect**: side effects (API calls, subscriptions) — runs after render. **useRef**: mutable ref that doesn't trigger re-render (also for DOM access). **useMemo**: memoize expensive calculations.

```
import { useState, useEffect, useRef, useMemo } from 'react';
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const renderCount = useRef(0); // doesn't cause re-render
  // useEffect: runs after render
  useEffect(() => {
    fetch(`/api/users/${userId}`) // side effect
    .then(r => r.json())
    .then(setUser);
  }, [userId]); // dependency array
  // useMemo: only recomputes when user changes
  const fullName = useMemo(() =>
    user ? `${user.first} ${user.last}` : '',
    [user]);
  renderCount.current++;
  return <div>Renders: {renderCount.current} | {fullName}</div>;
}
```

■ *Java parallel: useEffect with [] = @PostConstruct. useEffect cleanup = @PreDestroy. useMemo = @Cacheable. useRef = a mutable box that survives re-renders.*

QR5 What is the virtual DOM and how does React's reconciliation work?

Answer

The **virtual DOM** is an in-memory JavaScript representation of the real DOM. When state/props change, React builds a new virtual DOM tree, **diffs** it against the previous one (reconciliation), and applies only the minimal set of real DOM changes (patch).

React uses a **key** prop to identify list items uniquely — without keys, React may re-render or reorder incorrectly. Always use stable unique IDs, not array indices, as keys.

```
// Without key – React can't track which item changed
{items.map(item => <li>{item.name}</li>)} // ■ warning
// With key – React efficiently updates only changed items
{items.map(item => (
  <li key={item.id}>{item.name}</li> // ✓ correct
))}
// React's diff rules:
// 1. Different element type → destroy + recreate subtree
// 2. Same type → update attributes, recurse children
// 3. Lists with key → match by key, move/add/remove minimally
```

■ *Java parallel: Virtual DOM diffing ≈ Git diff. React patches only changed nodes — like Hibernate's dirty checking that only sends changed SQL updates.*

QR6 How does component communication work? (Parent↔Child, siblings, global state)

Answer

Parent → **Child**: pass props. **Child** → **Parent**: pass a callback function as prop, child calls it. **Siblings**: lift state to common parent. **Global/distant state**: Context API (built-in) or state management libraries (Redux, Zustand, Jotai).

```
// Parent → Child via props
function Parent() {
  const [count, setCount] = useState(0);
  return <Child count={count} onIncrement={() => setCount(c => c+1)} />;
}
// Child → Parent via callback prop
function Child({ count, onIncrement }) {
  return <button onClick={onIncrement}>Count: {count}</button>;
}
// Global state with Context
const ThemeCtx = React.createContext('light');
function App() {
  return (
    <ThemeCtx.Provider value='dark'>
      <DeepChild /> { /* can access theme without prop drilling */ }
    </ThemeCtx.Provider>
  );
}
function DeepChild() {
  const theme = React.useContext(ThemeCtx);
  return <div className={theme}>Content</div>;
}
```

■ *Java parallel: Context ≈ Spring's ApplicationContext / dependency injection. You provide a value high up; consumers access it anywhere below — no prop drilling.*

QR7 How do you fetch data from an API in React?

Answer

The standard pattern is **fetch inside useEffect**, with loading and error states. For production apps, use **React Query (TanStack Query)** or **SWR** — they handle caching, retries, background refresh, and loading states automatically.

```
// Basic fetch pattern
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    fetch('/api/users')
      .then(res => { if (!res.ok) throw new Error(res.status); return res.json(); })
      .then(data => setUsers(data))
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, []);
  if (loading) return <Spinner />;
  if (error) return <p>Error: {error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// Production: React Query
import { useQuery } from '@tanstack/react-query';
function UserList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(r => r.json()),
  });
  // automatic caching, retries, background refresh
}
```

■ *Java parallel: useEffect fetch ~ @PostConstruct calling a REST client. React Query ~ Spring Cache + RestTemplate combined, with automatic invalidation.*

QR8 What is event handling in React? How is it different from plain HTML?

Answer

React uses **synthetic events** — a cross-browser wrapper around native DOM events. Event names are **camelCase** (onClick, onChange, onSubmit). Handlers are functions, not strings. React attaches a single listener at the root (event delegation) for performance.

```
function Form() {
  const [value, setValue] = useState('');
  // camelCase handlers, pass function reference
  const handleChange = (e) => setValue(e.target.value);
  const handleSubmit = (e) => {
    e.preventDefault(); // prevent page reload
    console.log('Submitted:', value);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input
        value={value} // controlled input
        onChange={handleChange}
        placeholder='Type here'
      />
      <button type='submit'>Submit</button>
    </form>
  );
}
// HTML: onclick='handleClick()' - string
// React: onClick={handleClick} - function reference
```

■ *Java parallel: React's synthetic event $\approx$ `HttpServletRequest` — it wraps the native event and normalises across browsers.*

QR9 What is React Router and how does client-side routing work?

Answer

React Router enables navigation between views without full page reloads. It intercepts URL changes, matches routes, and renders the corresponding component — all in the browser. The server serves a single index.html (SPA).

```
import { BrowserRouter, Routes, Route, Link, useParams } from 'react-router-dom';
function App() {
  return (
    <BrowserRouter>
    <nav>
    <Link to="/">Home</Link>
    <Link to="/users">Users</Link>
    </nav>
    <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/users" element={<UserList />} />
    <Route path="/users/:id" element={<UserDetail />} />
    <Route path="*" element={<NotFound />} /> {/* 404 */}
    </Routes>
    </BrowserRouter>
  );
}
function UserDetail() {
  const { id } = useParams(); // extract :id from URL
  // fetch user by id...
}
```

■ *Java parallel: React Router ≈ Spring MVC @RequestMapping. Route path="/users/:id" ≈ @GetMapping("/users/{id}"). useParams() ≈ @PathVariable.*

QR1 How do you handle forms in React? Controlled vs uncontrolled components.

0

Answer

Controlled components: React state is the single source of truth for form values. Every keystroke updates state via `onChange`. **Uncontrolled components:** DOM holds the value; you read it via a ref when needed. Controlled is the React-idiomatic approach.

```
// CONTROLLED – state drives the input
function LoginForm() {
  const [form, setForm] = useState({ email: '', password: '' });
  const handleChange = (e) =>
    setForm(prev => ({ ...prev, [e.target.name]: e.target.value }));
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(form); // { email: '...', password: '...' }
  };
  return (
    <form onSubmit={handleSubmit}>
      <input name='email' value={form.email} onChange={handleChange} />
      <input name='password' value={form.password} onChange={handleChange} type='password' />
      <button>Login</button>
    </form>
  );
}

// UNCONTROLLED – DOM holds value, ref reads it
function SimpleForm() {
  const inputRef = useRef();
  const onSubmit = () => console.log(inputRef.current.value);
  return <><input ref={inputRef} /><button onClick={onSubmit}>Go</button></>;
}
```

■ *Java parallel: Controlled = data binding in Spring MVC model. React state = @ModelAttribute object updated on every field change.*

QR1 What are common React performance optimisations?

1

Answer

Key techniques: **React.memo** — skip re-render if props unchanged. **useCallback** — memoize function references (so child memo works). **useMemo** — memoize expensive computations. **lazy + Suspense** — code-split and load components on demand. **Virtualization** — render only visible rows for large lists.

```
import { memo, useCallback, useMemo, lazy, Suspense } from 'react';
// memo – skip re-render when props are same
const ExpensiveChild = memo(function ExpensiveChild({ data }) {
  return <div>{data}</div>;
});
function Parent() {
  const [count, setCount] = useState(0);
  // useCallback – stable fn reference across renders
  const handleClick = useCallback(() => {
    console.log('clicked');
  }, []); // recreated only if deps change
  // useMemo – expensive calc only on dep change
  const sorted = useMemo(() =>
    bigArray.sort((a,b) => a-b), [bigArray]);
  return <ExpensiveChild data={sorted} onClick={handleClick} />;
}
// Code splitting – load only when needed
const Dashboard = lazy(() => import('./Dashboard'));
function App() {
  return <Suspense fallback={<Spinner/>}><Dashboard /></Suspense>;
}
```

■ *Java parallel: React.memo = @Cacheable on component level. useCallback = preventing new lambda allocations on every call. Lazy = lazy bean initialisation.*

QR1 How does TypeScript improve React development?

2

Answer

TypeScript adds **static typing** to JavaScript, catching errors at compile time — the comfort zone for Java developers. In React: type props with interfaces, type state, type event handlers, and enjoy IDE autocomplete.

```
// Type your props with interface
interface User {
  id: number;
  name: string;
  email: string;
  role: 'admin' | 'user'; // union type = enum
}
interface UserCardProps {
  user: User;
  onDelete: (id: number) => void;
  className?: string; // optional prop
}
const UserCard: React.FC<UserCardProps> = ({ user, onDelete, className }) => {
  return (
    <div className={className}>
      <p>{user.name} ({user.role})</p>
      <button onClick={() => onDelete(user.id)}>Delete</button>
    </div>
  );
};

// Typed state and events
const [users, setUsers] = useState<User[]>([]);
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) =>
  setValue(e.target.value);
```

■ *Java parallel: TypeScript interface = Java interface/POJO. React.FC = generic type param. The ? optional modifier = @Nullable annotation.*

Java → Python → React Quick Reference

Concept	Java	Python	React / JS
Null check	<code>x != null</code>	<code>x is not None</code>	<code>x !== null / x !== undefined</code>
String format	<code>String.format("%s", x)</code>	<code>f"{x}"</code>	<code>`\${x}`</code> (template literal)
List create	<code>new ArrayList<>()</code>	<code>[]</code>	<code>[]</code> (same as JS)
Map create	<code>new HashMap<>()</code>	<code>{}</code>	<code>{}</code> (object literal)
Iteration	<code>for (T x : list)</code>	<code>for x in lst:</code>	<code>list.forEach / for...of</code>
Lambda	<code>(x) -> x*2</code>	<code>lambda x: x*2</code>	<code>(x) => x*2</code>
Stream/filter	<code>.stream().filter(p)</code>	<code>[x for x in l if p]</code>	<code>list.filter(p)</code>
Stream/map	<code>.stream().map(f)</code>	<code>[f(x) for x in l]</code>	<code>list.map(f)</code>
Async unit	<code>CompletableFuture</code>	<code>asyncio / async def</code>	<code>Promise / async/await</code>
Package mgmt	<code>Maven / Gradle</code>	<code>pip + requirements.txt</code>	<code>npm / package.json</code>
Dependency inj	<code>@Autowired Spring</code>	No built-in (use classes)	<code>React Context / Redux</code>
HTTP client	<code>RestTemplate / WebClient</code>	<code>requests / httpx</code>	<code>fetch / axios</code>
Print debug	<code>System.out.println(x)</code>	<code>print(x)</code>	<code>console.log(x)</code>
Ternary	<code>x>0 ? "pos" : "neg"</code>	<code>"pos" if x>0 else "neg"</code>	<code>x>0 ? 'pos' : 'neg'</code>
Try-catch	<code>try { } catch(E e) { }</code>	<code>try: ... except E as e:</code>	<code>try { } catch(e) { }</code>
Interface	<code>interface Flyable { }</code>	<code>Abstract class / Protocol</code>	<code>TypeScript interface</code>
Optional field	<code>@Nullable / Optional</code>	<code>Optional[T]</code> type hint	<code>field?: Type (TS)</code>